Dual-Domain MMD Adaptation: Consolidated Final Evaluation

July 31, 2026

1 System and training setup

1.1 Hardware

All runs in this document were trained on a single machine: 4× NVIDIA GeForce RTX 4090 (24 GB each, driver 535.171.04, CUDA 12.2), an AMD EPYC 74F3 24-core processor (96 threads, 2 sockets), and 503 GB system RAM. Every dual-domain adaptation and no-MMD fine-tune run used 2 of the 4 GPUs via ultralytics' native DDP (`--device 0,1`); pretraining baselines used a single GPU per run. Software: Ultralytics 8.4.104, PyTorch 2.5.1+cu121, Python 3.12.13.

1.2 Common hyperparameters

To keep the comparisons in this document meaningful, every run shares the same core training configuration unless stated otherwise: YOLOv10n, image size 1920 (native resolution, no upscaling), 200 max epochs with early-stopping patience 100, total batch size 16 (8 per GPU across 2 GPUs – three exceptions are noted per-section below), `optimizer=auto` (resolves to SGD), `lr0=0.01`, `momentum=0.937`, `weight_decay=0.0005`, 3 warmup epochs, and ultralytics' standard detection augmentation defaults (mosaic, HSV jitter, flips, etc., unchanged from stock). A few runs stopped early via the patience criterion (within 1–11 epochs of the 200 cap) rather than diverging; this is noted where relevant.

For the dual-domain MMD runs specifically, the following settings are shared across all variants: RBF kernel, EMA bandwidth with momentum 0.9, features extracted after detection layer index 10 (`mmd_target_layer=10`), flattened (not global-average-pooled) before the kernel. The MMD-specific knobs that vary between the four labeled variants used throughout this document are:

- **no_mmd**: plain fine-tune on the target domain only – no source batches, no MMD term at all.
- **naive_mmd**: constant `mmd_weight = 0.8` (script default), no bandwidth freeze – the un-regularized baseline.
- **reg_mmd**: linearly decaying `mmd_weight` plus a frozen kernel bandwidth after an early epoch, intended to prevent the bandwidth estimate collapsing before the target distribution has moved.
- **reg_mmd_smaller_weight**: the same regularization scheme as reg_mmd, decayed to a smaller final `mmd_weight`.

A note on hyperparameter provenance. Ultralytics' own `args.yaml` / checkpoint `train_args` do not capture the MMD-specific overrides (weight, schedule, freeze epoch) – they are stored on a nested config object that is not part of the serialized trainer args. The values above for `mmd_target_layer`, kernel, and preprocessing are certain (script defaults, never overridden across this study). The bandwidth-freeze-epoch used per comparison group is noted inline in each section below to the best of the project's own run-directory records; treat these as reliable but not double-logged the way the common hyperparameters above are.

2 Synthetic target: real_large_source_syn_small_target

Source = real_large (pretrained, detached), target = syn_small, imgsz 1920, evaluated on the syn val set. no_mmd/naive_mmd/reg_mmd ran at batch 16 on 2 GPUs; reg_mmd_smaller_weight ran at batch 12 on a single GPU (an unrelated scheduling choice, not expected to materially affect the comparison at this batch-size range). Bandwidth-freeze-epoch = 5 for both regularized variants.

Model	person mAP50	person mAP50-95	vehicle mAP50	vehicle mAP50-95
no_mmd	0.697	0.361	0.720	0.503
naive_mmd	0.674	0.344	0.693	0.480
reg_mmd	0.671	0.339	0.690	0.478
reg_mmd_smaller_weight	0.677	0.343	0.693	0.480

Table 1: real_large → syn_small direction, evaluated on syn val (target domain).

In this direction, no-MMD clearly beats every MMD variant on both classes – a real gap (2–3 points mAP50, 2–2.4 points mAP50-95), not the small margins seen elsewhere in this study. Regularization does not help here either; reg_mmd is marginally worse than naive_mmd. This replicates the direction-dependent asymmetry already noted in the project's own README results: source=real/target=synthetic improves over baseline, while source=synthetic/target=real does not fully close the gap – here, MMD actively underperforms plain fine-tuning in this direction.

3 Real target

Source = various synthetic pretrains, target = real_small (or real_small_vehicle_only), all evaluated on real val (target domain). Two comparisons follow.

3.1 ClearNoon-only source vs. all-weather source (syn_large)

Both groups use bandwidth-freeze-epoch = 1 for their regularized variants (matching hyperparameters for a fair comparison).

Source	Model	person mAP50	person mAP50-95	vehicle mAP50	vehicle mAP50-95
clearnoon	no_mmd	0.634	0.291	0.873	0.615
	naive_mmd	0.623	0.284	0.866	0.612
	reg_mmd	0.630	0.284	0.870	0.613
	reg_mmd_smaller_weight	0.617	0.284	0.868	0.612
syn_large	no_mmd	0.648	0.296	0.870	0.617
	naive_mmd	0.637	0.291	0.869	0.618
	reg_mmd	0.635	0.292	0.871	0.619
	reg_mmd_smaller_weight	0.639	0.291	0.871	0.617

Table 2: Single-weather (ClearNoon only) vs. all-weather (syn_large) synthetic source, both sized to 6400 train images.

syn_large (all four weather conditions) beats clearnoon-only on every single row for person, and on 3 of 4 rows for vehicle (tied/marginally behind on the 4th). The gaps are modest (0.5–1.4 points) but consistent in direction, suggesting weather diversity in the source domain – not just raw image count, since both sets are the same size – contributes a small but real benefit to target-domain performance.

3.2 Person diminishing under MMD, and the vehicle-only comparison

Same source dataset (syn_large), same regularization settings (bandwidth-freeze-epoch = 5) for the dual-class and vehicle-only rows, allowing a direct look at what happens to the person class as MMD strengthens, and what happens to vehicle when person is removed from training entirely.

Model	person mAP50	person mAP50-95	vehicle mAP50	vehicle mAP50-95
dual-class (person + vehicle)				
reg_mmd	0.642	0.294	0.872	0.618
reg_mmd_smaller_weight	0.635	0.289	0.876	0.620
vehicle-only (person removed entirely)				
reg_mmd	—	—	0.885	0.626
reg_mmd_smaller_weight	—	—	0.883	0.624

Table 3: Dual-class rows show person declining as MMD regularization strengthens (no_mmd → reg_mmd → reg_mmd_smaller_weight), while vehicle rises over the same sequence. Vehicle-only rows show vehicle detection with person's interference removed entirely.

Two things happen simultaneously as the MMD regularization gets stronger (no_mmd → reg_mmd → reg_mmd_smaller_weight): person mAP50 falls monotonically (0.648 → 0.642 → 0.635) while vehicle mAP50 rises monotonically (0.870 → 0.872 → 0.876). This is the clearest evidence in this study that MMD's small vehicle benefit comes at person's expense, not for free. Removing person from training entirely (vehicle-only) pushes vehicle mAP50 higher still, to 0.883–0.885 across all three variants – confirming the person class's noisier training signal was genuinely holding back vehicle-specific performance, consistent with the fitness-averaging mechanism discussed earlier in this study (person's volatility disproportionately affects which epoch gets selected as best.pt when the checkpoint metric averages both classes equally).

4 syn_xlarge vs. syn_large as source, to close out the study

Same target (real_small) and same regularization settings (mmd-weight 1.0 → 0.1, bandwidth-freeze-epoch = 5) throughout; only the source dataset's scale changes.

Source (# imgs)	Model	person mAP50	person mAP50-95	vehicle mAP50	vehicle mAP50-95
syn_large (6400)	no_mmd	0.648	0.296	0.870	0.617
	naive_mmd	0.637	0.291	0.869	0.618
	reg_mmd	0.642	0.294	0.872	0.618
	reg_mmd_smaller_weight	0.635	0.289	0.876	0.620
syn_xlarge (10240)	**reg_mmd**	0.635	0.291	**0.876**	**0.621**

Table 4: syn_xlarge (10240 images, all weathers) as source, same regularized MMD settings as syn_large's best-performing variant, both evaluated on real val. syn_xlarge's run used batch 8 on 2 GPUs (4 per GPU) rather than the batch-16 setup used elsewhere in this document.

Scaling the synthetic source from syn_large (6400 images) to syn_xlarge (10240 images) gives the best vehicle result of the entire study: 0.876/0.621, matching or edging out syn_large's own best regularized variant while leaving person roughly unchanged (0.635, identical to syn_large's smaller-weight variant). The gain is modest but consistent with the study's central premise – more of the cheap synthetic domain's data provides a small additional benefit on top of what the regularization already buys, without any additional cost beyond generating more synthetic frames.